

Newton Learn Hub: Assignment Submission Backend

Problem Statement

Build a backend for a simplified **Newton Learn Hub Assignment Portal** using **Node.js**, **Express**, **PostgreSQL**, and the **pg** package.

For this lab, you must use **raw SQL queries** to interact with PostgreSQL.

Database

Create a PostgreSQL database named:

`assignment_portal`

Create an `assignments` table with the following fields:

Field	Type	Description
<code>id</code>	SERIAL	Primary key
<code>title</code>	VARCHAR(200)	Assignment title
<code>deadline</code>	DATE	Assignment deadline
<code>submitted</code>	BOOLEAN	Whether the assignment is submitted

The `submitted` field should default to `false`.

Project Structure

Create the following project structure:

```
assignment-portal/  
| — node_modules/  
| — package.json  
| — app.js  
| — db.js
```

| — .env

Install the required packages:

```
npm install express pg dotenv
```

Database Configuration

Create a `.env` file containing your PostgreSQL configuration:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=yourpassword
DB_NAME=assignment_portal
```

Use these environment variables when creating your PostgreSQL connection.

Requirements

1. PostgreSQL Connection

Create `db.js` and configure a PostgreSQL connection using the `pg` package.

The database credentials must be read from environment variables.

Use a PostgreSQL connection pool.

2. Express Server

Create an Express application in `app.js`.

- Use `express.json()` to parse JSON request bodies.
 - Start the server on port `3000`.
-

3. Create Assignment

Create the following API:

POST /assignments

The request body will contain:

```
{
  "title": "Backend Lab",
  "deadline": "2026-08-10"
}
```

Insert the assignment into the PostgreSQL database.

The `submitted` value should use its database default.

Return the newly created assignment in the response.

The response status should be:

201

Important

Use a **parameterized SQL query** when inserting the assignment.

4. View All Assignments

Create:

GET /assignments

Return all assignments stored in the database.

Example response:

```
[
  {
    "id": 1,
    "title": "Backend Lab",
    "deadline": "2026-08-10",
```

```
"submitted": false
}
]
```

Challenge

Modify the query so that assignments are returned with the **newest assignment first**.

Hint:

ORDER BY id DESC

5. Mark Assignment as Submitted

Create:

PATCH /assignments/:id

For example:

PATCH /assignments/1

The API should update the assignment with the given **id** and set:

submitted = true

Return the updated assignment in the response.

Use a **parameterized SQL query**.

6. Delete an Assignment

Create:

DELETE /assignments/:id

For example:

DELETE /assignments/1

Delete the assignment with the given `id`.

The response should contain a success message and the deleted assignment.

Example:

```
{
  "message": "Assignment deleted successfully",
  "assignment": {
    "id": 1,
    "title": "Backend Lab",
    "deadline": "2026-08-10",
    "submitted": true
  }
}
```

If the assignment does not exist, return:

404

with:

```
{
  "message": "Assignment not found"
}
```

Requirement

Use PostgreSQL's:

RETURNING *

in the `DELETE` query so that the deleted assignment can be returned in the response.

7. Filter Submitted Assignments

Create support for the following request:

GET /assignments?submitted=true

When `submitted=true` is provided, return only assignments whose `submitted` value is `true`.

Example:

```
[
  {
    "id": 2,
    "title": "Express Lab",
    "deadline": "2026-08-12",
    "submitted": true
  }
]
```

Use the query parameter:

`req.query.submitted`

and a parameterized SQL query.

Final APIs

Your application should support:

```
POST /assignments
GET /assignments
PATCH /assignments/:id
DELETE /assignments/:id
GET /assignments?submitted=true
```

Submission Checklist

Before submitting, verify that:

- PostgreSQL connection works.
- All required APIs are implemented.
- Data is stored in PostgreSQL.
- Parameterized queries are used where user input is involved.
- **PATCH** updates **submitted** to **true**.
- **DELETE** returns **404** when the assignment does not exist.
- **DELETE** returns the deleted assignment.
- **GET /assignments?submitted=true** returns only submitted assignments.
- **GET /assignments** returns newest assignments first.

Important: Commit your code after completing each route.

As soon as you successfully complete and test the **first route (POST /assignments)**, make your first Git commit.

Continue committing your code after completing each subsequent route.